

Program Synthesis Using Inductive Logic Programming for the Abstraction and Reasoning Corpus

Intelligenza Artificiale
2025, Vol. 19(2) 85–101
© The Author(s) 2025



Article reuse guidelines:
sagepub.com/journals-permissions
DOI: 10.1177/117248035251363178
journals.sagepub.com/home/ina

Sage | IOS Press

Filipe Marinho Rocha^{1,2} , Inês Dutra^{1,3} , Vítor Santos Costa^{1,2} and
Luís Paulo Reis^{4,5}

Abstract

The Abstraction and Reasoning Corpus (ARC-AGI) is an Artificial General Intelligence benchmark that is currently unsolved. It demands strong generalization and reasoning capabilities, which are known to be weaknesses of Neural Network based systems. In this work, we propose a Program synthesis system to solve it, which casts an ARC-AGI task as a sequence of Inductive Logic Programming tasks. We have implemented a simple Domain Specific Language that corresponds to a small set of object-centric abstractions relevant to the benchmark. This allows for adequate representations to be used to create logic programs, which provide reasoning capabilities to our system. When solving each task, the proposed system can generalize from a few training pairs of input-output grids. The obtained logic programs are able to generate objects present in the output grids and can transform the test input grid into the output grid solution. We developed our system based on some ARC-AGI tasks that do not require more than the small number of primitives that we implemented and showed that our system can solve unseen tasks that require different reasoning.

Keywords

Program synthesis, ILP, Logic Programming, ARC, AGI

Received: February 7, 2025; accepted: July 14, 2025

Introduction

Machine Learning (ML) (Carbonell et al., 1983), and specifically Deep Learning (DL) (LeCun et al., 2015), has achieved great success and even surpassed human performance in a number of important tasks (Shrestha & Mahmood, 2019). These successes are in what is called narrow Artificial Intelligence (AI), where each DL model is designed to solve a specific task very well. However, current DL models are known to lack generalization capabilities and do not solve different kinds of task (Lake et al., 2017; Marcus, 2018).

Artificial Neural Network (ANN) performance is known to be greatly reduced when applied to Out-of-Distribution (OOD) data (Farquhar & Gal, 2022; Kirchheim et al., 2024; Ye et al., 2022). Large Language Models (LLMs) (Naveed et al., 2023) have shown impressive abilities, shortening the gap between Machine and Human Intelligence. However, they still show a lack of reasoning capabilities (Guo et al., 2023; McCoy et al., 2023; Nezhrina et al., 2024; Wu et al., 2023).

The ARC-AGI challenge (Chollet, 2019) was designed by François Chollet to provide a measure of progress in

AGI (Artificial General Intelligence) (Adams et al., 2012; Bubeck et al., 2023). The challenge was first proposed in 2019, but it remains unsolved, even using state-of-the-art models, such as LLMs (Bober-Irizar & Banerjee, 2024; Lee et al., 2024).

ARC-AGI is composed of a Public Training set with 400 tasks and a Public Evaluation set with 400 tasks. There is also a test set with 100 tasks, which is completely private. These tasks require a variety of reasoning to solve, and an AI system developer cannot anticipate all the

¹Faculdade de Ciências da Universidade do Porto, Porto, Portugal

²INESCTEC, Campus da FEUP, Porto, Portugal

³CINTESIS@RISE-Health, Porto, Portugal

⁴Faculdade de Engenharia da Universidade do Porto, Rua Dr. Roberto Frias, Porto, Portugal

⁵LIACC, FEUP, Porto, Portugal

Corresponding Author:

Filipe Marinho Rocha, Faculdade de Ciências da Universidade do Porto, Rua do Campo Alegre, 4169-007 Porto, Portugal and INESCTEC, Campus da FEUP, Rua Dr. Roberto Frias 4200 - 465 Porto, Portugal.

Email: up199803145@up.pt

reasoning skills required to overcome the challenge. As Chollet describes, ARC-AGI requires developer-aware generalization, which is stronger than OOD generalization (Chollet, 2019).

ARC-AGI can be considered as a benchmark for Program Synthesis (Chollet, 2019). This is a subfield of AI (Gulwani et al., 2015; Gulwani, Polozov, Singh, & et, 2017) with the purpose of generating programs that satisfy a high-level specification, often provided in the form of example pairs of inputs and outputs for the program, which is exactly the ARC-AGI format, as can be seen in Figure 1. The logic behind each task is very different, and the relations between objects are key to the solutions.

Inductive Logic Programming (ILP) (Muggleton & De Raedt, 1994) is an ML method that performs program synthesis (Cropper et al., 2022) and is known to be able to learn and generalize from a few training examples (Lin et al., 2014; Muggleton et al., 2018), which is the case with ARC-AGI (median of 3 training examples for each task). To the best of our knowledge, before our work, ILP has never been applied to the ARC-AGI challenge. Most ILP systems synthesize First-Order Logic (FOL) programs in Prolog (Körner et al., 2022).

We propose a program synthesis system, Induce logic programs for Abstraction and Reasoning (ILPAR), that uses ILP on top of a small set of object-centric abstractions. It does program synthesis by searching for relations between objects in the task examples and describing these in FOL, which is a very natural way to encode relations. It returns a program that is composed by a sequence of logic programs that are capable of generating objects in an empty output grid, in order to reach the solution.

The object-centric abstractions are defined in a Domain Specific Language (DSL) that we created for ARC-AGI. It is a small and limited DSL that can represent just some of the types of objects and relations present in the dataset. We selected 10 tasks that our DSL can tackle to develop ILPAR in order to solve them in an abstract and general way that can generalize to unseen tasks.

Related Work

The ARC-AGI challenge was launched in 2020 on Kaggle, a popular online platform that hosts ML competitions, and at the time, the best solutions relied on the search for discrete programs, such as the winning entry for 2020 (Chollet et al., 2025), which used 142 manually crafted DSL functions and brute-force program search, to achieve 20% on the private test set.

Despite the advent of LLMs (e.g., GPT-3, 3.5, 4), early attempts to use these systems to beat ARC-AGI were unsuccessful. GPT-4V, which is GPT4 enhanced for visual tasks (Achiam et al., 2023), was also far from solving it (Mitchell et al., 2023; Singh et al., 2023; Xu et al., 2023).

Recent advances came from tailoring LLMs to ARC-AGI particularities, such as in the form of improved DSLs (Hodel, 2024), which increased the efficiency of program search. Progress accelerated during the ARC-AGI Prize 2024, catalyzed by three major categories of approaches (Chollet et al., 2025):

- Deep learning-guided program synthesis (Chen et al., 2018): particularly, using specialized code LLMs, to generate solution programs or guide the program search process beyond brute-force program search
- Test-time training (TTT) for transductive models (Akyürek et al., 2024): fine-tuning an LLM at training time on a given task in order to adapt the base LLM to the task at hand
- Combining program synthesis together with transductive models (Li et al., 2024): the two approaches above into a single approach, where it was observed that each approach tends to solve different kinds of tasks

Transductive models produce the output grid directly without generating code that executes transformation steps to the grid, as do inductive program synthesis methods.

The 2024 ARC-AGI challenge edition featured two leaderboards. In addition to Kaggle's competition leaderboard based on the Private evaluation set, where the final results of the 2024 edition are shown in Table 1, the ARC-AGI challenge 2024 edition also featured a secondary leaderboard based on the Public evaluation set and its final results are shown in Table 2.

The Public leaderboard was hosted outside Kaggle and its participants were allowed internet access and relaxed computing constraints for the purpose of evaluating closed-source frontier models. Submissions were evaluated on the Public evaluation set but also on the Semi-private evaluation set, a new dataset with 100 new tasks created for this purpose, where solutions were not shared with the participants but training example pairs and test input grids were.

The state-of-the-art consists of solutions based on LLMs Akyürek et al. (2024); Bonnet and Macfarlane (2024); Chollet et al. (2025); Li et al. (2024), which populate both leaderboards, but there exists different research outside of the challenge, which explores the use of symbolic AI methods and is more closely related to our proposal.

There are approaches to ARC-AGI based on Dreamcoder (Alford et al., 2021; Banburski et al., 2020; Ellis et al., 2021), a neuro-symbolic system for inductive programming synthesis. The latest is called PeARL (Bober-Irizar & Banerjee, 2024). This approach does not generate logic programs, but functional ones. Since the tasks present logic reasoning challenges, it seems that logic programs should be more appropriate for the problem at hand.

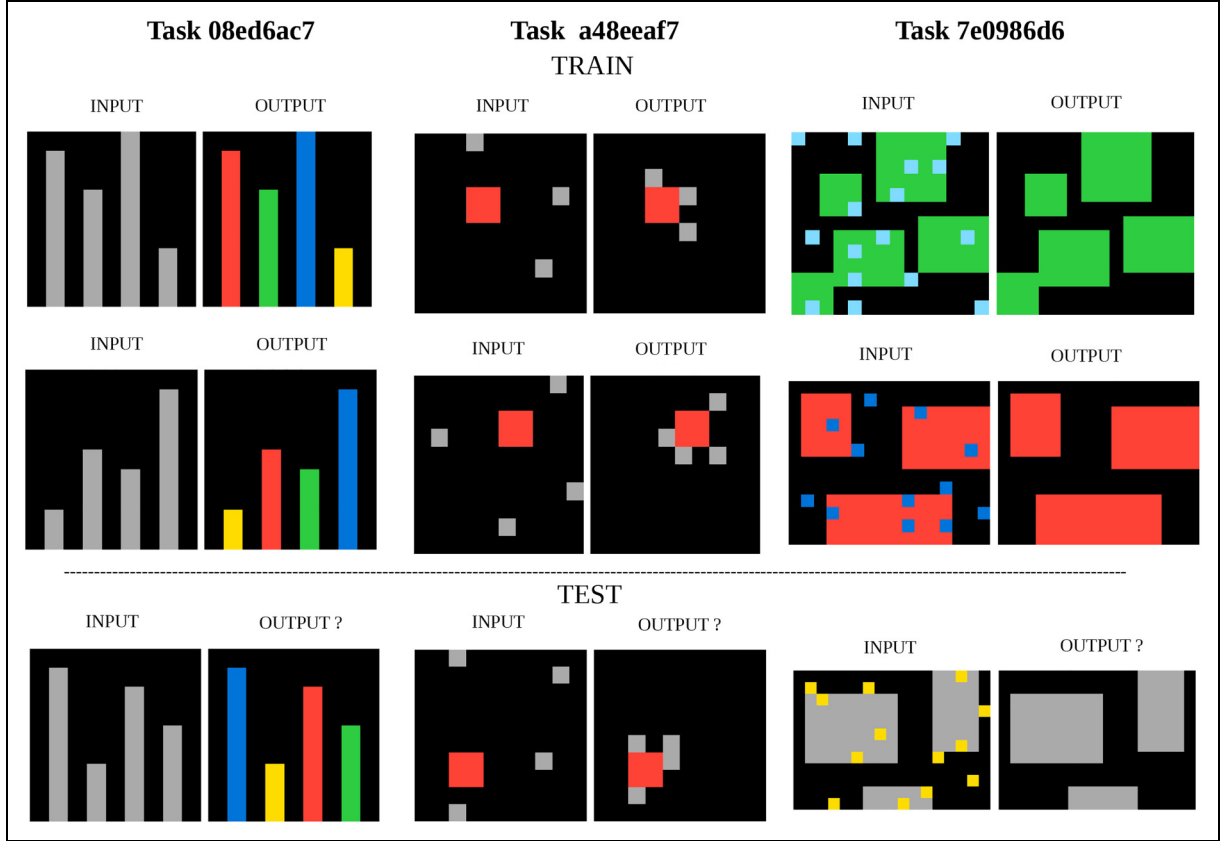


Figure 1. Example Tasks of the ARC-AGI Training Dataset With the Solutions Shown. The Goal is to Produce the Test Output Grid Given the Test Input Grid and the Train Input-Output Grid Pairs. These Tasks Refer to Very Different Concepts: Color Coding by Object Size Ranking, Moving Some Kind of Object Towards Another Kind of Object, and Denoising Objects.

Table 1. Top Performers of the Private Leaderboard in the ARC-AGI 2024 Edition and Winners of the ARC Prize (Chollet et al., 2025).

Name	Score (Private Evaluation Set)
The ARCHitects Li et al. (2024)	53.5%
Guillermo Barbadillo Akyürek et al. (2024)	40%
alijs Bonnet and Macfarlane (2024)	40%
William Wu	37%
PoohAI	37%

There is a system based on analogical reasoning applied to ARC-AGI, called BEN (Witt et al., 2023). It learns functional programs, not logic programs, despite using object-centric abstractions and decomposing the tasks by objects and their relations, as we propose, as well.

There is ARG (Xu et al., 2022), an object-centric approach that uses graphs grounded in FOL to encode relations between objects. Our system also uses FOL to encode relations.

GP (Lei et al., 2024) casts an ARC problem as a generalized planning (GP) problem, where a solution is formalized as a planning program. It expresses each ARC problem using the standard Planning Domain Definition Language (PDDL)

Table 2. Top Performers of the Public Leaderboard in the ARC-AGI 2024 Edition Chollet et al. (2025).

Name	Score (Semi-Private Eval)	Score (Public Eval)
Jeremy Berman	53.6%	58.5%
Akyürek et al. LLM+ICL+TTT	47.5%	62.8%
Ryan Greenblatt	43%	42%
...	...	...
OpenAI o1-preview (pass@1)	18%	21%
Anthropic Claude 3.5 Sonnet (pass@1)	14%	21%
OpenAI GPT-4o (pass@1)	5%	9%
Google Gemini 1.5 (pass@1)	4.5%	8%

coupled with external functions representing object-centric abstractions. PDDL uses a limited set of FOL.

More recently and after our initial work, another ILP based approach to ARC appeared, using POPPER (Hocquette & Cropper, 2025), a modern all-around ILP system. This approach also uses task decomposition but without object-centric abstractions. Input and output grids are decomposed into sets of input and output pixels, which seems very limiting for an ILP system to work with.

Regarding state-of-the-art concepts and methods, like Deep Learning-guided program synthesis, these can be combined with Symbolic AI as it is the case with DreamCoder. In the case of Test-time training, it is a concept already part of ILP, where a logic program (model), is created (trained), based on training examples given to solve a particular task, while relying on base knowledge encoded in a logic program (base model), also known as Background Knowledge (BK).

Developing specific DSLs to tackle ARC-AGI showed to be crucial in the case of LLMs and we also developed a specific DSL for ARC-AGI. Our approach can be easily extended by applying DL to guide program search.

ILP

Inductive Logic Programming (ILP) is a form of logic-based ML. The goal is to induce a hypothesis, in the form of a logic program or set of logical rules, given training examples and BK.

As with other forms of ML, the goal is to induce a hypothesis that generalizes the training examples. However, while most forms of ML use tensors to represent data and learn functions, ILP uses logic programs and learns relations.

Preliminaries

A FOL language L is a tuple:

$$(P, A, F, V),$$

where P is a set of predicates, A is a set of constants, F is a set of function symbols (functors), and V is a set of variables. Using L we can construct terms, atoms or literals, and clauses.

A term is a constant, a variable, or a compound term: $f(u_1, \dots, u_m)$ consisting of a functor and terms as arguments. An atom is a formula $p(t_1, \dots, t_n)$, where p is an n -ary predicate symbol and $t_1, \dots, t_n$ are terms. Terms in a program represent objects while predicates represent relations among these objects.

A ground atom is an atom without variables. We refer to ground atoms as facts. An atom is a positive literal and a negation of an atom is a negative literal.

A clause is a finite disjunction of literals (positive or negative). A definite clause is a clause with exactly one positive literal. If $A, B_1, \dots, B_n$ are literals, then $A \vee \neg B_1 \vee \dots \vee \neg B_n$ is a definite clause. It can also be written in the implication form: $A \Leftarrow (B_1 \wedge \dots \wedge B_n)$. We refer to definite clauses as rules and we write them in the Prolog format: $A :- B_1, \dots, B_n$.

The atom A is called the head, and the set of negative literals $B_1, \dots, B_n$ is called the body. We refer to the negative literals that compose the body as just literals. A definite program is a set of definite clauses or rules. We refer to a definite program as a logic program.

Problem Setting

We use the Learning-from-Entailment setting of ILP (Raedt, 2008). In this setting, the ILP task is to take as input:

$$(E^+, E^-, B, L),$$

where:

- E^+ are the positive examples of a target predicate, in the form of facts
- E^- are the negative examples of a target predicate, in the form of facts
- B is the BK, previous knowledge in the form of facts and rules
- L is the FOL language,

and to return H : a hypothesis contained in L , in the form of a logic program or rule(s), that together with the BK, entails most of the positive and least of the negative examples.

We use an entailment-based cost function to guide the hypothesis search. Our cost function also takes into account the hypothesis size, and abstraction and generality metrics, we developed. A optimal hypothesis corresponds to a hypothesis with the lowest cost.

A hypothesis can be a single rule or multiple rules. Often, a single rule is insufficient to describe a target concept, so we allow for multiple rules.

Also, reaching a logical definition of a complex relation may require obtaining a very large program, that is, one composed of many rules or many literals in the rule(s). Learning a large program with ILP is very challenging, but there are some approaches that try to overcome this, such as divide-and-conquer (Cropper, 2022).

Sometimes negative examples of a target predicate are not given. Positive-only learning has been widely studied in ILP. One approach to this issue, is to generate negative examples.

The fundamental ILP problem is to efficiently search a large hypothesis space in order to build a logic program. There are ILP approaches that search in a top-down, bottom-up or hybrid fashion.

Divide-and-Conquer

Divide-and-conquer approaches in ILP divide the set of examples into disjoint subsets and search for a program for each subset. We use a divide-and-conquer approach, but instead of applying it to examples, we divide each individual example into sets of objects and relations present in the example, as in previous work on ARC-AGI (Witt et al., 2023).

Instead of searching for a program that transforms an entire input image to an entire output image, we search for logic programs that can generate subsets of the output image, that is, can generate one or more objects present in the output image.

So, we decompose a task into simpler relational synthesis subtasks. Specifically, our representation decomposes a training input-output example into a set of input objects, a set of output objects and a set of relations between these.

Positive-Only

After retrieving the set of input-output relations between objects, we have the positive examples for each target predicate. We reason under the closed-world assumption (Reiter, 1978) to generate negative examples. To obtain the negative examples, for each target predicate:

$$p(V_1, \dots, V_n),$$

and for each value w_{ij} in the domain of V_i , we generate all grounded relations:

$$p(w_{1j}, \dots, w_{nj}),$$

where the ones that are not part of the set of positive examples are considered the negative examples.

Top-Down

In a top-down approach, one starts from the most general rule, that covers all the examples: positive and negative, iteratively adding literals to its body, to cover more positive and less negative examples, as long as it does not become overly specific. In a bottom-up approach, one would start from a long rule that may cover just one positive example, iteratively removing literals from it, until it becomes overly general.

We use a top-down approach to clause building, inspired by FOIL (Quinlan, 1990).

DSL for ARC-AGI

Chollet recommends starting with developing a DSL capable of expressing all possible solution programs for any ARC-AGI task. Since the exact set of ARC-AGI tasks could be anything that involves Core Knowledge priors (Chollet, 2019), this is a challenge. Object awareness is considered one of the Core Knowledge prior of humans,

necessary to solve ARC-AGI tasks (Acquaviva et al., 2022; Chollet, 2019; Johnson et al., 2021) and is central to general human visual understanding (Spelke & Kinzler, 2007).

Object-centric abstractions substantially reduce the search space by enabling the focus on the relations between objects, rather than individual pixels or irrelevant pieces of the grids. We devised a DSL composed of object-centric abstractions that form the BK required for the ILP task.

Objects and Relations Between Objects

We have manually defined a simple DSL in FOL, that can represent: grid, pixel, object and some relations between objects, defined in Table 3. An object is ultimately defined by an ordered list of contiguous pixels. From a list of pixels, the rest of the object variables can be obtained.

The *arc_obj* predicate has arguments such as Color. This corresponds to the most frequent color, since an object can contain different colored pixels. Clean is the percentage of noise or different colors relative to the most frequent color. An object with Clean=100 is totally composed of pixels of the same color. An object can be of type: point, line, rectangle, or undefined.

The *project*, *translate*, *copy*, and *copy_with_change_pixel* predicates correspond to generative relations in which given an object and other variables as input, they can generate other objects. The relation described by the *objects_relation* predicate is different since it does not generate objects but returns properties of the relation between two objects, like, for example: distance between objects, or, in which direction is one object relative to another.

XdirTouch and YdirTouch are the shortest distances in the X and Y axes to move an object in order to touch another object. Xdir and Ydir are the shortest distances required to move an object to take another one's place in the grid, if they are of the same shape and size, and if not, to move inside the space of the other.

Our DSL only covers some of the Core Knowledge Priors as shown in Table 4.

Multiple Representations

An image representation in our object-centric approach is defined by a list of objects and a background color. This

Table 3. DSL Logical Predicates.

Predicates	Arguments
grid	(Xdim, Ydim, Bkcolor)
pixel	(Xcoord, Ycoord, Color)
arc_obj	(PixelsLst, ObjType, Size, Xmin, Xmax, Ymin, Ymax, Color, Clean)
objects_relation	(Obj1, Obj2, XdirTouch, YdirTouch, Xdir, Ydir, Orientation, Direction, Distance)
project	(Obj, Orientation, Direction, Len, NewColor, OutputObj)
translate	(Obj, Xdir, Ydir, Direction, OutputObj)
copy	(Obj, NewClean, NewColor, OutputObj)
copy_with_change_pixel	(Obj, Color0, Color1, Color2, OutputObj)

Table 4. Coverage of the Core Knowledge Priors by Our DSL.

Core Prior	Subprior	Coverage
Objectness	Object cohesion	Yes
	Object persistence	Yes
	Object influence via contact	Yes
Goal-directedness		Yes
Numbers and Counting		No
Basic Geometry and Topology	Object shapes	Yes
	Symmetries, rotations, translations	No
	Shape upscaling / downscaling, elastic distortions	No
	Containing / being contained / being inside or outside of a perimeter	No
	Drawing lines, connecting points, orthogonal projections	Yes
	Copying, repeating objects	Yes

representation can build a target image from an empty grid by filling the grid with the background color and then drawing each object of the list on top of it.

An image grid can be defined by multiple object representations. For example, a single rectangle in an empty grid can also be defined as several contiguous Line or Point objects.

Since we do not know the logic behind the transformation of the input grid into the output grid, we do not know which object representations will be more relevant to the solution; therefore, we keep multiple possibly overlapping object representations for the same image.

In Figure 2, we can see an example of, depending on the input-output transformation logic required, one particular object description is more appropriate than the other. In this case, all transformations may require only one relation: *copy*, and vary only on the object selected as the input: a point, a line, or a rectangle.

Likewise, the same image transformation can be explained by different relations. Looking at the example in Figure 2, if we want to explain the input-output 1 transformation, we have *copy* applied to the rectangle, which is the simplest explanation. But *project* applied to the two top points in the rectangle, with *length=1* and *direction=down*, would build the same rectangle in the output.

Since there may be multiple relations that explain an image transformation that may intersect in some space of the output image, that is, they generate objects in the output grid that may overlap, we keep multiple relation representations over the same grid space.

Since ARC-AGI tasks are made up of multiple training examples, the simplest explanation for an individual example could be different from the one common to multiple examples. So, we work with multiple and intermingled representations of objects and relations until we get the final program(s) that can reach the solution.

ILPAR

ILP is usually seen as a method for concept learning but is capable of building a logic program in Prolog (Körner et al., 2022), which is Turing-complete, hence it performs program synthesis. We extend this, by combining logic programs to build a bigger program that generates objects in sequence, to fill an empty grid, which corresponds to, procedurally, applying grid transformations to reach the solution, as shown in Figure 9.

A relation can be used to generate objects. As mentioned before, all relations in our DSL, except for the *objects_relation*, are able to generate objects given the first object of the relation. To generate a particular output, the relation should be constrained by a logic program.

Example of a logic program in Prolog that defines the application of *project*:

```
target(arc_obj(Color,...),Orientation,
Direction,Len,Color1,OutputObj):-
    arc_obj(Color,...),
    (... variables constraints ...)
    equal(Len,4),
    equal(Orientation,vertical),
    equal(NewColor,Color),
    project(arc_obj(Color,...),Orientation,
Direction,Len,NewColor,OutputObj).
```

All variables in the target relation are constrained, so, for example, *Orientation* can take as values: vertical, horizontal, up-diagonal, and down-diagonal, and *Color*: one of 10 colors.

The literal: *arc_obj(...)*, in the rule, is instantiated with the facts that were added to the BK describing the objects present in the input grid: the red and light blue points, shown in Figure 3. This logic program can generate unambiguously two lines in the output. Given the *Orientation=vertical*, the variable *Direction* is not necessary,

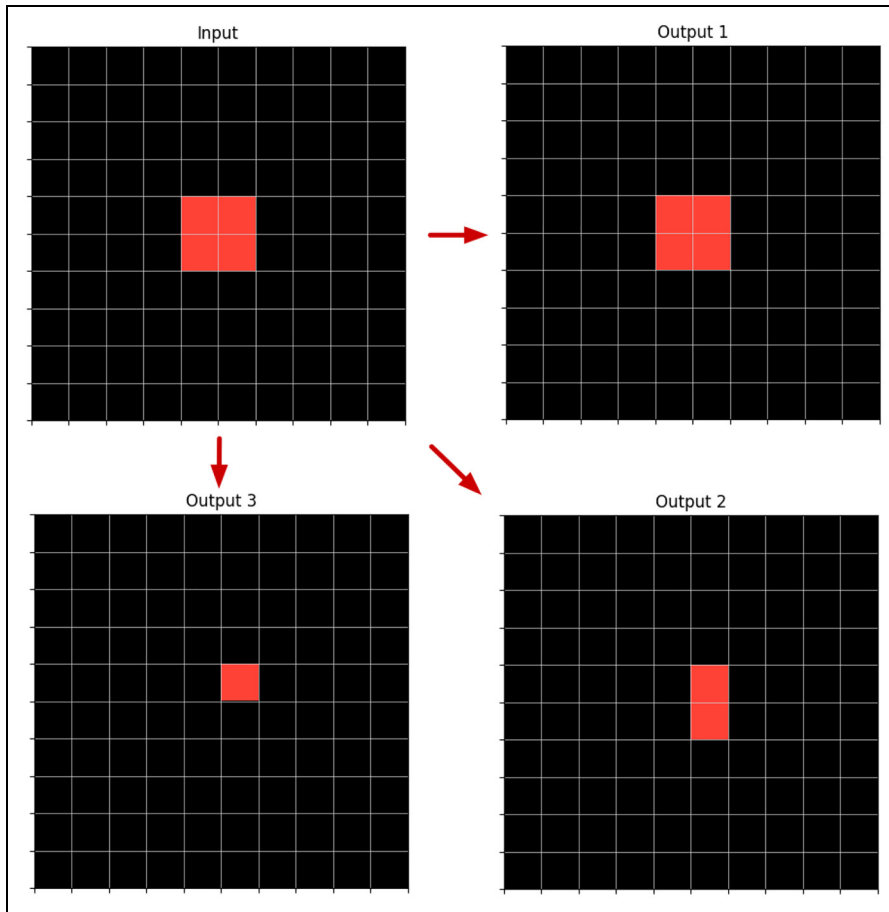


Figure 2. Example of An Input Grid with a Rectangle and Three Possible Output Grids Depending If *Copy* is Applied to the Full Rectangle or Just a Line or a Point Inside It.

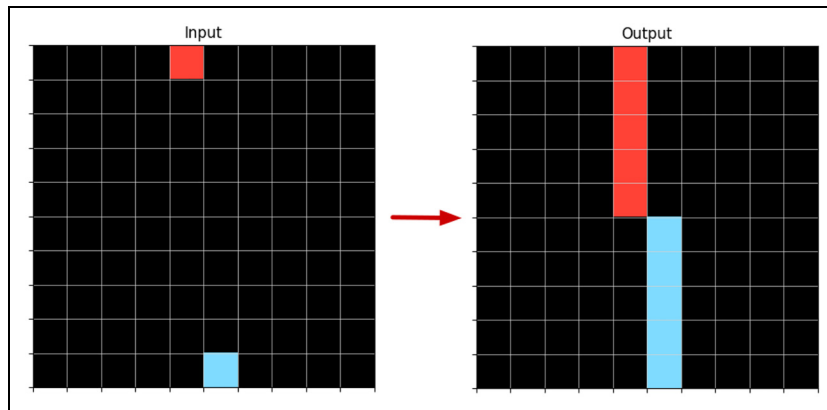


Figure 3. Example of An Input-Output Transformation When Applying a *Project* Logic Program Which is Able to Generate Lines From Points, in This Case, Vertical Lines with Length 4, and with the Same Color as the Point. Note that the Input Points are Copied as Well Using a *Copy* Program.

since the points are on the edges of the grid and so, can only grow into lines in one direction each: down and up.

If we would remove the literal that instantiates the variable *Orientation* from the body of the rule, it would generate multiple lines from each input point, as shown in Figure 4.

So, by obtaining logic programs through ILP, we are indeed constructing a program that generates objects that can fill an empty test output grid, in order to reach the solution. We can see an overview of ILPAR in Figure 5.

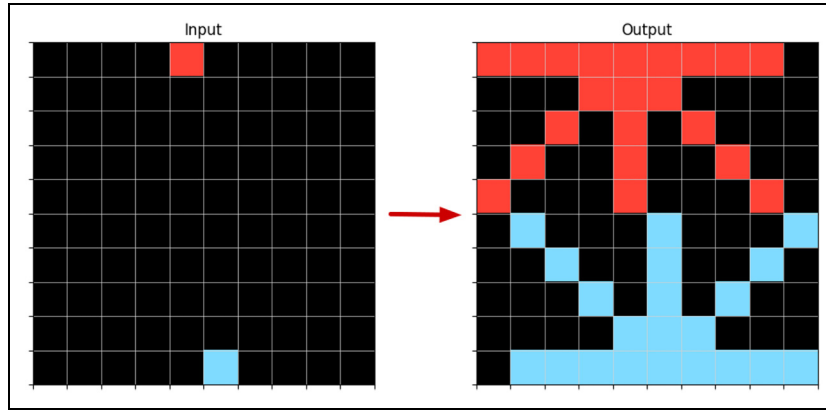


Figure 4. Example of an Input-Output Transformation When Applied a *Project Logic Program* With Length 4 But Without Having the Orientation Defined. Note That the Input Points are Copied as Well Using a *Copy Program*.

Note that ILPAR does not use meta-level ILP strategies obtained from learning from several solved tasks, it simply performs task-specific ILP.

Objects and Relations Retrieval

Our system begins by retrieving all objects present in the input and output grids, for each example of a task.

Then, we search for the relations that hold between the retrieved objects. We search for relations between objects present in the input grid: input-input relations, and between objects in the input grid and output grid: input-output relations.

We store all of the grounded objects and relations retrieved, as facts in the BK to be used by ILP.

For the training example in Figure 6, the grounded objects retrieved in the input grid are:

```
arc_obj([ pixel(3,0,pink)] ,point,...)
arc_obj([ pixel(1,1,green)] ,point,...)
arc_obj([ pixel(2,1,green)] ,point,...)
arc_obj([ pixel(4,1,green)] ,point,...)
arc_obj([ pixel(1,1,green),pixel(2,1,
green)] ,line,...)
arc_obj(pixel(0,3,blue)] ,point,...)
arc_obj(pixel(2,3,blue)] ,point,...)
arc_obj(pixel(2,4,red)] ,point,...)
```

Some of the grounded relations present in the input grid are:

```
objects_relation(arc_obj([ pixel(3,0,
pink)] ,...),down_margin,1,0,vertical,
down,1)
objects_relation(arc_obj([ pixel(3,0,
pink)] ,...),down_margin,1,1,
down_diagonal,down_right,1)
```

```
objects_relation(arc_obj([ pixel(3,0,
pink)] ,...),arc_obj([ pixel(2,1,
green)] ,...),
0,0,up_diagonal,up_right,0)
objects_relation(arc_obj([ pixel(3,0,
pink)] ,...),arc_obj([ pixel(4,1,
green)] ,...),
1,1,down_diagonal,down_right,0)
objects_relation(arc_obj([ pixel(4,1,
green)] ,...),down_margin,0,0,vertical,
down,0)
objects_relation(arc_obj([ pixel(4,1,
green)] ,...),down_margin,0,0,
down_diagonal,down_right,0)
objects_relation(arc_obj([ pixel(4,1,
green)] ,...),down_margin,0,0,up_diagonal,
down_left,0)
```

Some of the input-output grounded relations are:

```
translate(arc_obj([ pixel(3,0,pink)] ,
point,...),1,0,down,arc_obj([ pixel(4,0,
pink)] ,point,...))
translate(arc_obj([ pixel(2,1,green)] ,
point,...),1,0,down,arc_obj([ pixel(3,1,
green)] ,point,...))
translate(arc_obj([ pixel(1,1,green)] ,
point,...),1,0,down,arc_obj([ pixel(2,1,
green)] ,point,...))
translate(arc_obj([ pixel(1,1,green)] ,
point,...),2,0,down,arc_obj([ pixel(3,1,
green)] ,point,...))
translate(arc_obj([ pixel(1,1,green) ,
pixel(2,1,green)] ,line,...),1,0,down,
arc_obj([ pixel(2,1,green),pixel(3,1,
green)] ,line,...))
translate(arc_obj([ pixel(1,1,green) ,
pixel(2,1,green)] ,line,...),2,0,down,
```

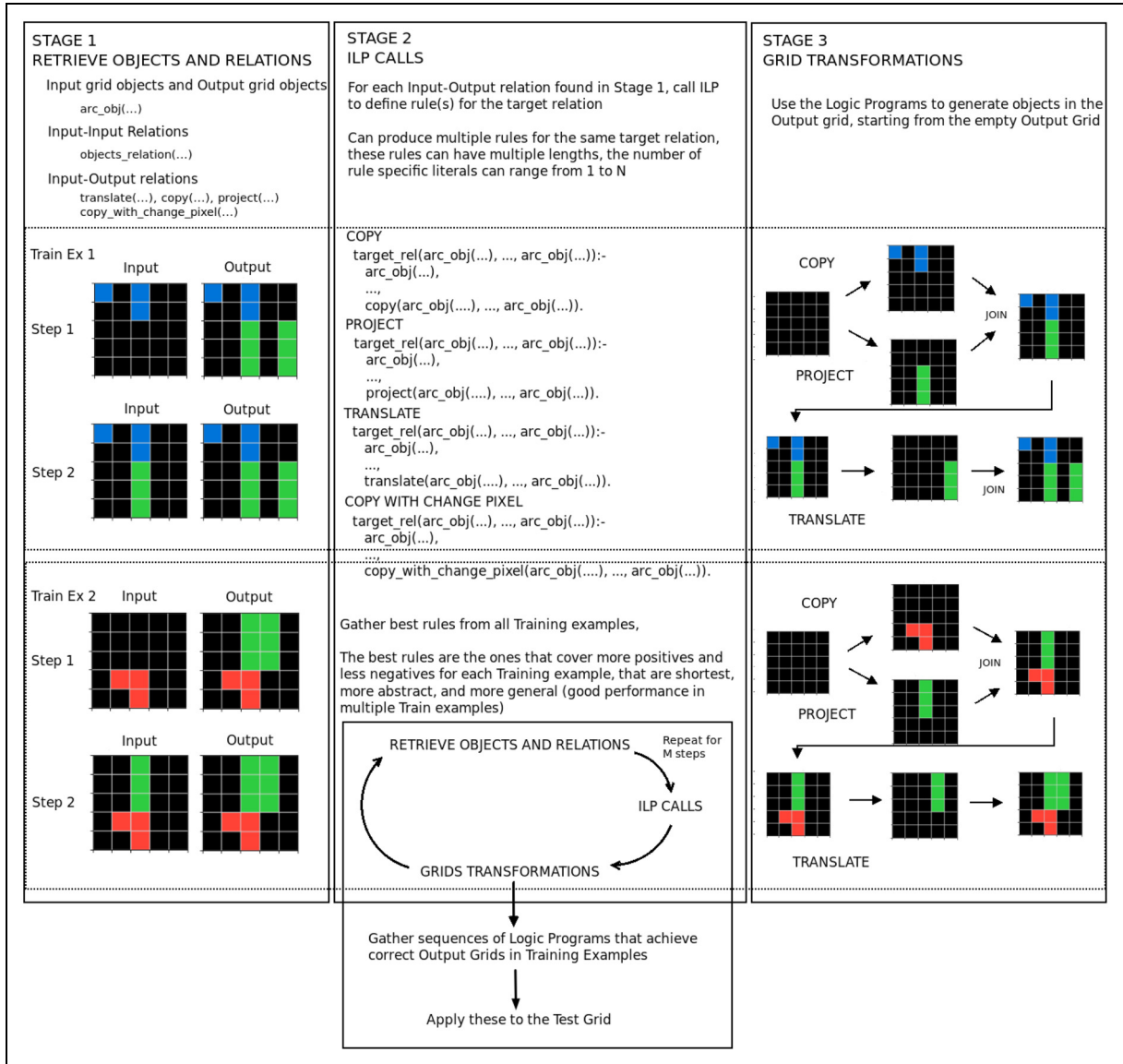


Figure 5. ILPAR System Overview.

```
arc_obj([ pixel(3,1,green), pixel(4,1,
green)], line, ...)

copy(arc_obj([ pixel(4,1,green)],
point, ...), 100, green, arc_obj([ pixel(4,1,
green)], point, ...))
copy(arc_obj([ pixel(2,1,green)],
point, ...), 100, green, arc_obj([ pixel(2,1,
green)], point, ...))
project(arc_obj([ pixel(4,1,green)],
point, ...), up, 2, green,
arc_obj([ pixel(3,1,green), pixel(2,1,
green)], line, ...))
project(arc_obj([ pixel(4,1,green)],
point, ...), up, 1, green, arc_obj([ pixel(3,1,
```

```
green)], point, ...))
project(arc_obj([ pixel(2,1,green)],
point, ...), down, 2, green,
arc_obj([ pixel(3,1,green), pixel(4,1,
green)], line, ...))
project(arc_obj([ pixel(1,1,green)],
point, ...), down, 1, green,
arc_obj([ pixel(2,1,green)], point, ...))

project(arc_obj([ pixel(2,1,green)],
point, ...), down, 1, green,
arc_obj([ pixel(3,1,green)], point, ...))
project(arc_obj([ pixel(1,1,green)],
point, ...), down, 3, green,
```

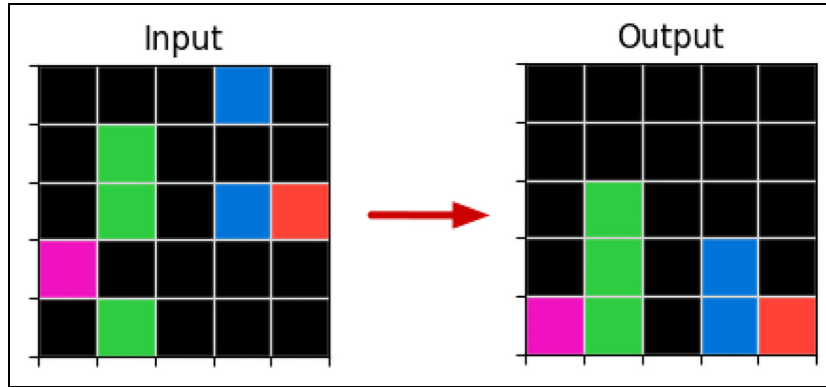


Figure 6. Third Training Example of the Task 1e0a9b12. The Concept of Gravity Can Explain This Task. Using Our DSL It Can Be Described as: Translate Objects Down Until They Touch the Object Below or the Bottom Margin of the Grid, and Copy Objects If They are Already Touching the Bottom Margin.

```
arc_obj([pixel(2,1,green),pixel(3,1,
green),pixel(4,1,green)],line,...)
```

ILP Calls

We then call ILP to create logic programs that describe, in an abstract way, the relations found in the previous step.

Input-input relations can be part of the body of the rules but cannot correspond to target relations to be defined by a logic program, since they do not generate any object in the output grid.

In the training example in Figure 6, ILPAR starts by defining the application of *translate*, since this is the relation that covers the most space in the example, so it is the target predicate in this case. The hypothesis search starts with the most general rule:

```
target(arc_obj(...), Xdir, Ydir, Direction,
OutputObj):-
  arc_obj(PixelsLst,ObjType,Size,Xmin,
Xmax,Ymin,Ymax,Color,Clean),
  member(Xdir,[ -4,-3,-2,-1,0,1,2,3,4]),
  member(Ydir,[ -4,-3,-2,-1,0,1,2,3,4]),
  member(Direction,[ left,right,up,down,
up_right,down_left,up_left,down_right]),
  (...body of the rule to be constructed...)

  translate(arc_obj(...), Xdir, Ydir,
Direction, OutputObj).
```

In ILPAR, the head of the rule is always composed of the predicate *target* and the terms of the target predicate as its arguments. This is a special predicate created for the goal query where the output of this query gives the coverage of the rule: positive and negative examples of the target predicate that it covers and an entailment-based cost function is applied to these values.

The target predicate itself with its arguments is placed at the end of the rule. The first literal in the body of the rule corresponds to the input object. The next literals in the rule correspond to the constraints.

Constraints are defined for each of the variables of the target predicate. In this case, constraints are defined for the variables: *Xdir*, *Ydir*, and *Direction*. *Direction*, *Orientation* and *Color* constraints are the same for any grid, but for this specific 5×5 grid, both *Xdir* and *Ydir* can only take values in the interval: $[-4, 4]$.

The relations and objects returned by the ILP call are added to the BK and taken now as if they were present in the initial input data.

We keep the logic programs being created and the updated versions of the output grid states after applying these programs to generate objects. The first ILP call receives as input: empty output grid states, but then with each subsequent ILP call, the updated output grid states are taken as input, where to build upon. This process can be seen in Figure 5.

Candidate Generation. The candidate literals to be added to the body of the rule in the ILP task, can be formed by the *grid*, *arc_obj* and *objects_relation* predicates defined in Table 3 and extra predicates that are part of Prolog: *member/2*, *equal/2*, *greater_than/2*, *lower_than/2*. The last two can only relate numeric variables. We also provide *linear_combination/4*, which consist of $Y = aX + b$ formulations as candidate literals, X and Y being numeric variables and a and b , numeric constants in some interval we pre-define.

As we mentioned before, the literals corresponding to the input object and the constraints are placed at the beginning of the rule and the literal with the target predicate at the end of the rule. This corresponds to the most general rule for a specific target predicate and train example, and so,

candidate generation for the hypothesis search, just focus on the rest of the body of the rule.

Candidate literals are added to the most general rule and coverage of the resulting rules is measured to evaluate their quality. ILPAR also evaluates rules quality based on their size and abstraction and generalization characteristics.

Since we are using objects and relations with typed variables, we only generate those candidates that relate each of the variables of the target relation and of its input object, to variables or constants of the same type.

Positive and Negative Examples. As we have seen in Section “ILP”, the usual approach to hypothesis building requires positive and negative examples. However, the ARC-AGI dataset is made up only of positive examples. So we created a way to get negative examples, as described in Section “Positive-only”.

In our system, for each logic program being constructed, the positive examples are the relations that the program generates that exist in the training data, and the negative examples are the relations that a program generates but do not exist in the training data.

For example, consider that the output in Figure 3 is the correct one, but the program generates all the lines in the output of Figure 4, like the shorter clause we mentioned. For this program, the number of positives covered would be 2: the lines in Figure 3 and the number of negatives covered would be 8: all the lines in Figure 4, except the vertical ones.

Here, our object-centric approach also reduces the complexity of the problem, compared with taking the entire grid into account, to define what is a positive example or not.

Matching Between Training Examples. A single ILP call is made for a target relation taking into account all of the task’s training examples. Our ILP system is designed to prefer logic programs that match between examples, that is, the same program produces a good coverage when applied to different examples. This is what we call generalization ability and we devised a metric to measure it that is part of the cost function that guides hypothesis search.

ILPAR should return logic programs abstract enough to generalize to all of the train examples. But in practice, this is not required. One of the development tasks we selected, shown in Figure 7, demonstrated that finding a program that solves all training examples may be unnecessary. In this case a program that generates vertical lines matches between the first two train examples and a program that generates horizontal lines matches between the last two.

If we wanted a solution that matched between all the train examples, described in natural language, it would be something like: draw lines from points that reach the opposite border of the grid and then translate these lines repeatedly in the perpendicular direction of the lines until the end of the grid. Finding the program that satisfies all 4

examples would be more complex than finding a solution that matches only between the 2 first examples.

Rules, Grid States and Search

As we mentioned before, the fundamental problem in ILP is to efficiently search in a large hypothesis space. A popular ILP approach is to use a set covering algorithm to learn hypotheses one rule at a time. Systems that implement this approach are often efficient because they are example-driven (Cropper et al., 2022). Our system works in this way.

Each ILP call produces a rule that defines the way a target relation generates objects and returns new output grid states with these objects added, for each train example, as shown in Figure 5.

For each step of the training grids transformations, several ILP calls are made, one for each target relation. ILP calls outputs are placed on the given output grid states and using the generated objects and updated input-input relations as the new BK for the next step of ILP calls. A simple breadth-first search is used to find the complete program(s) that can solve the training examples, in one or more steps.

In the training example shown previously in Figure 6, the first ILP call produces the following rule:

```
target(arc_obj(PixelsLst,ObjType,Size,
Xmin,Xmax,Ymin,Ymax,Color,Clean),
  Xdir,Ydir,Direction,OutputObj):-
  arc_obj(PixelsLst,ObjType,Size,Xmin,
Xmax,Ymin,Ymax,Color,Clean),
  (... variables constraints ...)

objects_relation(
  arc_obj(PixelsLst,ObjType,Size,Xmin,
Xmax,Ymin,Ymax,Color,Clean),
  arc_obj(PixelsLst1,ObjType1,Size1,
Xmin1,Xmax1,Ymin1,Ymax1,Color1,Clean1),
  XdirTouch,YdirTouch,Xdir1,Ydir1,
Orientation,Direction1,Distance),
  equal(Direction1,down),
  equal(Direction,Direction1),
  equal(Xdir,XdirTouch),
  equal(Ydir,YdirTouch),
  translate(arc_obj(PixelsLst,ObjType,
Size,Xmin,Xmax,Ymin,Ymax,Color,Clean),
  Xdir,Ydir,Direction,OutputObj).
```

This rule could be expressed in natural language as: move objects down to touch other objects (including the bottom margin object). This rule does not have a perfect score, that is, it does not cover all positive *translate* relations and no negative one, but it is the rule with the best

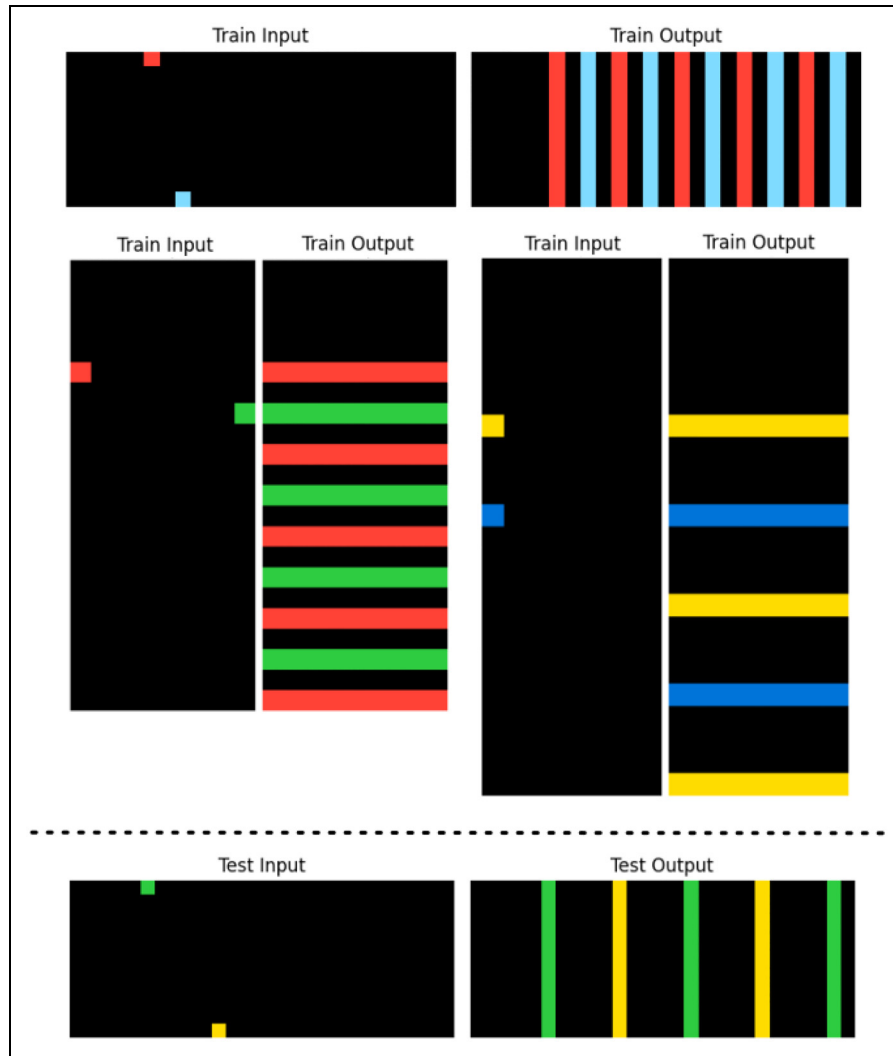


Figure 7. Example Task 0a938d79 That Contains Two Train Examples With Vertical Lines and Two Train Examples With Horizontal Lines, in the Output Grids. The Test Example Only Requires Vertical Lines in the Same Way as the First Two Train Examples. A Program That Solves the First Two Train Examples, Works Correctly on the Test Example.

score from all the candidates and it matches well between training examples.

After applying this rule, theoretically we would have two possible output grid states: output grid 1 and output grid 2, in Figure 8.

But in practice, since we do not allow for intersections, the two blue points cannot move to the same place, and so the second transformation is considered invalid and discarded in the search.

Since the valid output grid 1 does not correspond to the correct output grid, another application of the same rule or another ILP call to find a new rule, is required. In this case, after applying the same rule again on output grid 1 as input, the top blue point moves down to the top of the other, and the correct solution is found, as shown in Figure 8.

The search is based on finding the right sequence of logic programs that can build the train output grids, starting from

empty output grids, as shown in Figure 9. When we have at least one program that solves two or more train examples and can also produce a valid solution in the test output grid, we consider it a valid program and a candidate solution.

A valid program is one that builds a consistent theory. A program that in any way it can be applied to a grid, generates objects that intersect with each other, is inconsistent. We consider programs like this invalid and discard them in our search.

At each step of the program search, as shown in Figure 5, ILPAR discards the invalid programs and saves, from the partial solutions of train examples, the best rules - the ones with the following characteristics:

- better coverage: that cover more positives and less negatives

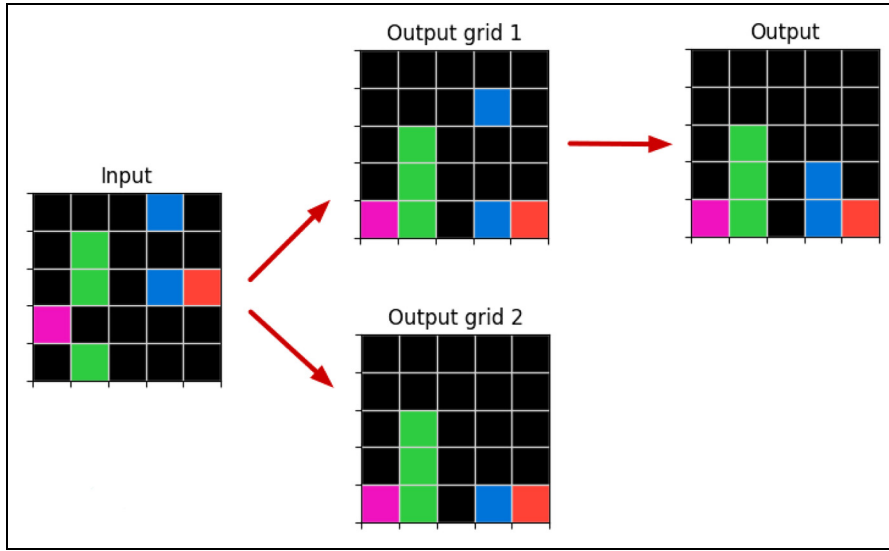


Figure 8. Solving Training Example From Task 1e0a9b12. The Same *Translate* Rule Could Produce Two Different Output Grids. To Get Output Grid 2 Both the Blue Points Were Moved to the Same Cell Which Our System Doesn't Allow, so the Transformation to Output Grid 1 is Chosen. To Get to the Correct Output Grid it is Required Another Application of the Same Rule.

- more abstract: with more predicates and variables and less constants in the body of the rule(s)
- more general: that work effectively within more train examples
- shorter: with less rules and rules with less literals and shorter literals.

In the program search, multiple sequences of logic programs that can solve the train examples are obtained and these will be put to the test when applied to the test grid, and only the ones that produce a consistent solution in the test grid are chosen as potential solutions to the task. The shortest sequences of the best rules are chosen from these. If there is still a tie, the final solution is chosen randomly.

Note that ILPAR would just need to reduce the number of output solutions to two, since the ARC-AGI challenge accepts two candidate solutions, and, if one is correct, a full score is given to the submission.

Deductive Search

In Figure 10, we can see an example task that in order to be solved, it would be easier to do it in reverse, that is, the input grid being generated from the output grid, but since we cannot solve the test example in this way, we use a form of deductive search to overcome this.

When ILPAR applies a logic program obtained by ILP calls, there can be different outputs depending on the order of object generation. When an object is generated in the output grid, no other object generated afterwards can intersect with it.

So, the coverage of grid space of the same logic program may vary, depending on the order of the program

application. It is the procedural aspect of our system. When applying several logic programs in sequence, this problem gets even bigger.

When applying the full program to produce the test output grid, we use a deductive search to find the way of applying it that covers the most surface. Since the final program can reproduce the train output grid(s), using this method, we should also have a solution that can produce the correct test output grid, using the same method.

Experimental Results

ILPAR was developed based on 10 tasks that only require the small DSL we developed - 9 of the tasks are part of the Public training set and 1 of the Public evaluation set. ILPAR, unsurprisingly, can solve the 10 tasks used for development and was designed to do so. The development tasks and its solutions are presented in the Supplemental Appendix. All solutions are presented in Prolog (the constraints literals were removed from the rules for better readability).

In order to evaluate the system, we needed to challenge it with unseen tasks, even by the developer of the system. Due to time-of-computing constraints, we did not apply ILPAR to the full 400 tasks of the Evaluation dataset. We randomly selected 20 tasks from this dataset to test our system (after removing the one evaluation task used to develop it). ILPAR solved 6 of these, demonstrating 30% accuracy on this sample.

We configured ILPAR for the experimental evaluation in the following way: for ILP calls, we set a maximum length of 5 literals for the body of each rule (besides the literals

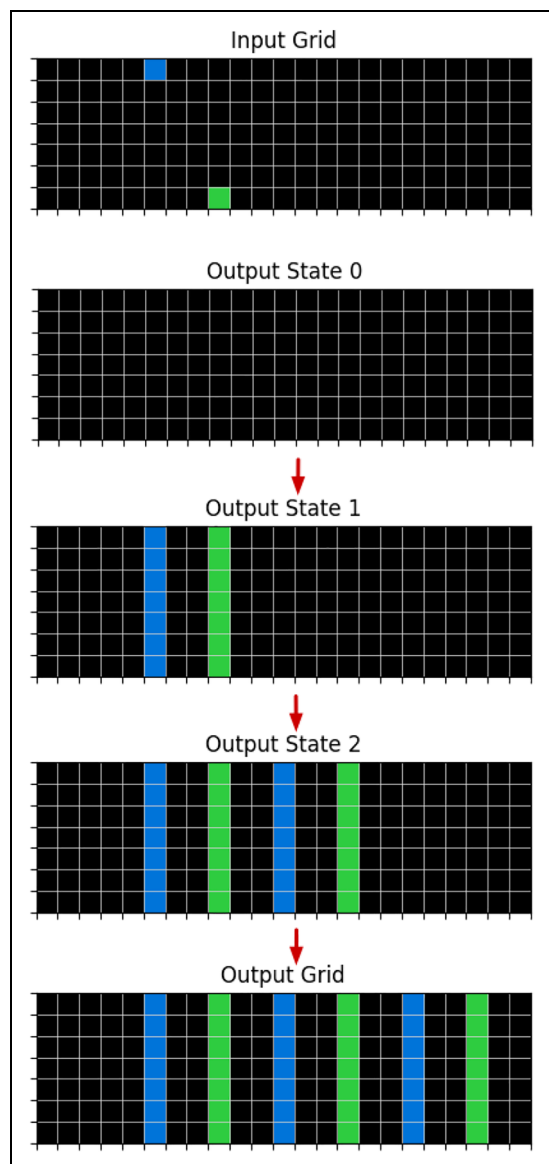


Figure 9. Example of a Successful Sequence of Transformations to the Output Grid State Reaching the Final State, Which is the Correct Output Grid in One of the Train Examples of the Task 0a938d79.

corresponding to the input object, variables constraints and target predicate) and a maximum of 3 rules for each target predicate at each step of the search. For the program search, we set a maximum of 3 steps, that is, a complete program can have a maximum of 3 logic programs in sequence.

Each task took an average of 9 hours to be solved. We used a single core of an AMD Ryzen 9 5900HX CPU with 16GB of RAM.

In the Supplemental Appendix, we present all the evaluation tasks and the output solutions for each successfully solved task, in Prolog.

Discussion

Some of the tasks not solved by ILPAR, require Core priors (Chollet, 2019) that are not part of its DSL, like symmetry awareness or concept of inside/outside. One of the tasks requires more than 3 logic programs in sequence, so our configuration of a maximum of 3 steps for the program search, could have been the limiting factor.

Since ILPAR was developed primarily on the basis of tasks that are considered easy: Training set, it is possible that it is not fully prepared to tackle tasks with a significant increase in complexity: Evaluation set (Chollet et al., 2025).

We found out there are some redundancy between ARC-AGI tasks, as some tasks in our Evaluation sample, required similar logical concepts as some tasks in our Training/Development set, such as sorting objects by the number of their holes, which could have made it easier for ILPAR to solve them.

It is interesting to note that ILPAR is capable of sorting objects by some of its variables without being explicitly programmed to do so. There is no sorting function in our DSL. It shows that simple and common primitives of ILP, such as *greater_than*, together with our object-centric representation, is capable of more than what the developer explicitly injected into the system.

Since the top 5 solutions on the Private evaluation set have a performance of 37% or higher, and the top 3 solutions on the Public evaluation set have a performance of 42% or higher, as can be seen in Tables 1 and 2, respectively, we consider the estimated performance of 30% of ILPAR to be very positive in comparison, especially because we can further develop the DSL and extend the maximum number of steps in the program search to increase performance.

We can compare ILPAR with the other (Neuro) Symbolic AI systems using the performance on the dataset we used to evaluate ILPAR. ILPAR's performance in the Evaluation set which is considered the hard dataset (as the Private evaluation set) is superior to the reported accuracies of the other systems in the Training set which is considered the easy dataset, which can be seen in Table 5.

There is a downward trend in performance from the Training subset (very easy) to the Training fullset (easy), and to the Public evaluation set (hard). So, except for the state-of-the-art solution LLM+ICL+TTT, in the first place, it seems our system demonstrates the best performance.

GPAR has good performance in the Training subset but is not evaluated in any other dataset. We cannot conclude much from this, since this subset is not representative of the full Training dataset, as the decreasing performances of the systems tested in both, show.

In the time-to-computing criteria, our solution is the worst, but very different computing resources were used, which makes the comparison elusive in this regard. We used a

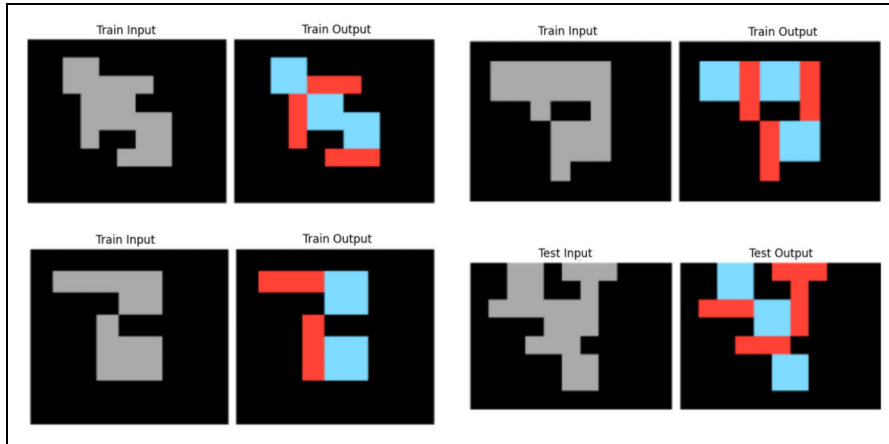


Figure 10. Example Task 150deff5 Where the Output Grids are More Informative Than the Input Grids, to Define the Object Relations. Deductive Search is Essential for This Task. The Red Lines and Blue Squares Have to be Placed in the Way that Covers the Most Space: The Same Grey Space of the Input Grid.

Table 5. Performance (Accuracy) of State-of-the-Art, the First Solution in the Public Evaluation Leaderboard: LLM+ICL+TTT; ILPAR Related Systems: POPPER, ARG, GPAR, BEN, PeARL and Our System.

System	Training Subset	Training Fullset	Public Evaluation Set	Avg time
LLM+ICL+TTT	NA	NA	62,8%	7 min
POPPER	NA	22%	NA	60 min
ARG	35,62%	11,5%	NA	3 min
GPAR	50,63%	NA	NA	30 min
BEN	33,75%	22,5%	NA	<1 min
PeARL	NA	16,5%	4,5%	60 min
ILPAR	NA	NA	30%*	540 min

Note: Performance on the Training subset of 160 object-centric ARC tasks, on the full Training set and on the Public evaluation set. Average time to reach a solution for each task is also presented. Note that each system used very different computing resources. *Estimated - performance on a random sample of 5% of the Public Evaluation Set.

Python implementation of ILPAR that has a lot of room to be improved in terms of speed and we plan to do so. With a faster implementation we plan to evaluate our system on a bigger sample to be more confident in the obtained results.

Since ILPAR is a completely different approach from state-of-the-art, it is possible that an approach that combines both kinds of methods can surpass the current best levels of performance on the benchmark. It should be investigated if ILPAR is good at a certain type of tasks and LLMs based solutions at others, which if it were the case, the potential for a productive combination should exist.

Conclusion

ILP is at the core of our system, and we showed that by providing it with only a small DSL, it is able to learn from a few training examples given for each ARC-AGI task and find the logic behind it. ILP gives our system: abstract learning and reasoning, and generalization capabilities, which is at the core of the ARC-AGI challenge.

The 30% performance on unseen tasks that have low redundancy with the tasks used for the development of

the system, shows that ILPAR has abstraction, reasoning and generalization abilities.

Since the rest of the ARC-AGI tasks may depend on many other DSL primitives, we plan to develop our DSL to at least be able to represent all human core priors identified in Chollet (2019). We plan to investigate a way to automate DSL creation from a given domain by using Predicate Invention (Cropper et al., 2022). This should be important to expand our DSL but also when dealing with different domains, not just a 2D grid environment, without the need to hand-craft the DSL, as we did in this work.

We plan to improve the speed of the ILPAR implementation and then apply our system to the entire Public and Semi-Private evaluation datasets to gather more insight about our approach and use the data obtained from it, to train a Deep Learning model to guide program search and increase its efficiency.

With a more efficient ILPAR, we plan to apply it to the Private Dataset, which requires tighter time-of-computation constraints, and we expect to obtain a similar performance obtained on our small sample, since the hardness of the

Private dataset is considered on par with the Public Evaluation dataset. With an extended DSL, we expect to achieve a higher performance.

Acknowledgments

This work is financed by National Funds through the Portuguese funding agency, FCT -Fundação para a Ciência e a Tecnologia, within project UIDB/50014/2020.

ORCID iDs

Filipe Marinho Rocha  <https://orcid.org/0000-0003-4171-4459>

Inês Dutra  <https://orcid.org/0000-0002-3578-7769>

Vítor Santos Costa  <https://orcid.org/0000-0002-3344-8237>

Luís Paulo Reis  <https://orcid.org/0000-0002-4709-1718>

Funding

The authors received no financial support for the research, authorship and/or publication of this article.

Declaration of Conflicting Interests

The authors declared no potential conflicts of interest with respect to the research, authorship, and/or publication of this article.

Supplemental Material

Supplemental material for this article is available online.

References

- Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altschmidt, J., Altman, S., Anadkat, S., et al. (2023). Gpt-4 technical report. arXiv preprint arXiv:2303.08774.
- Acquaviva, S., Pu, Y., Kryven, M., Sechopoulos, T., Wong, C., Ecanow, G., Nye, M., Tessler, M., & Tenenbaum, J. (2022). Communicating natural programs to humans and machines. *Advances in Neural Information Processing Systems*, 35, 3731–3743.
- Adams, S. S., Arel, I., Bach, J., Coop, R., Furlan, R., Goertzel, B., Hall, J. S., Samsonovich, A., Scheutz, M., Schlesinger, M., Shapiro, S. C., & Sowa, J. F. (2012). Mapping the landscape of human-level artificial general intelligence. *AI Mag*, 33, 25–42. <https://api.semanticscholar.org/CorpusID:2373342>
- Akyürek, E., Damani, M., Qiu, L., Guo, H., Kim, Y., & Andreas, J. (2024). The surprising effectiveness of test-time training for abstract reasoning. <https://arxiv.org/abs/2411.07279>
- Alford, S., Gandhi, A., Rangamani, A., Banburski, A., Wang, T., Dandekar, S., Chin, J., Poggio, T., & Chin, P. (2021). Neural-guided, bidirectional program search for abstraction and reasoning. <https://arxiv.org/abs/2110.11536>
- Banburski, A., Gandhi, A., Alford, S., Dandekar, S., & Chin, S. (2020). tomaso a poggio: Dreaming with ARC. In: Learning Meets Combinatorial Algorithms at NeurIPS2020. <https://openreview.net/forum?id=gjy2V1ko6t>
- Bober-Irizar, M., & Banerjee, S. (2024). Neural networks for abstraction and reasoning: Towards broad generalization in machines. arXiv preprint arXiv:2402.03507.
- Bober-Irizar, M., & Banerjee, S. (2024). Neural networks for abstraction and reasoning: Towards broad generalization in machines. <https://arxiv.org/abs/2402.03507>
- Bonnet, C., & Macfarlane, M. V. (2024). Searching latent program spaces. <https://arxiv.org/abs/2411.08706>
- Bubeck, S., Chandrasekaran, V., Eldan, R., Gehrke, J. A., Horvitz, E., Kamar, E., Lee, P., Lee, Y. T., Li, Y. F., Lundberg, S. M., Nori, H., Palangi, H., Ribeiro, M. T., & Zhang, Y. (2023). Sparks of artificial general intelligence: Early experiments with gpt-4. *ArXiv abs/2303.12712*. <https://api.semanticscholar.org/CorpusID:257663729>
- Carbonell, J. G., Michalski, R. S., & Mitchell, T. M. (1983). An overview of machine learning. *Machine Learning*, 1, 3SEP–23.
- Chen, X., Liu, C., & Song, D. X. (2018). Execution-guided neural program synthesis. In *International Conference on Learning Representations*. <https://api.semanticscholar.org/CorpusID:53317540>
- Chollet, F. (2019). On the measure of intelligence. arXiv preprint arXiv:1911.01547.
- Chollet, F., Knoop, M., Kamradt, G., & Landers, B. (2025). Arc prize 2024: Technical report. <https://arxiv.org/abs/2412.04604>
- Cropper, A. (2022). Learning logic programs though divide, constrain, and conquer. In *Proceedings of the AAAI Conference on Artificial Intelligence* (vol. 36, pp. 6446–6453).
- Cropper, A., Dumančić, S., Evans, R., & Muggleton, S. H. (2022). Inductive logic programming at 30. *Machine Learning*, 111(1), 147–172.
- Ellis, K., Wong, C., Nye, M., Sablé-Meyer, M., Morales, L., Hewitt, L., Cary, L., Solar-Lezama, A., & Tenenbaum, J. B. (2021). Dreamcoder: bootstrapping inductive program synthesis with wake-sleep library learning. In: *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. p. 835–850. PLDI 2021, Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3453483.3454080>
- Farquhar, S., & Gal, Y. (2022). What ‘out-of-distribution’ is and is not. In *NeurIPS ML Safety Workshop*.
- Gulwani, S., Hernández-Orallo, J., Kitzelmann, E., Muggleton, S. H., Schmid, U., & Zorn, B. (2015). Inductive programming meets the real world. *Communications of the ACM*, 58(11), 90–99.
- Gulwani, S., Polozov, O., Singh, R., & et, al (2017). Program synthesis. *Foundations and Trends® in Programming Languages*, 4(1-2), 1–119.
- Guo, Z., Jin, R., Liu, C., Huang, Y., Shi, D., Yu, L., Liu, Y., Li, J., Xiong, B., Xiong, D., et al. (2023). Evaluating large language models: A comprehensive survey. arXiv preprint arXiv:2310.19736.
- Hocquette, C., & Cropper, A. (2025). *Relational decomposition for program synthesis*. <https://arxiv.org/abs/2408.12212>
- Hodel, M. (2024). Arc-dsl: A domain-specific language for solving arc tasks. <https://github.com/michaelhodel/arc-dsl>

- Johnson, A., Vong, W. K., Lake, B. M., & Gureckis, T. M. (2021). Fast and flexible: Human program induction in abstract reasoning tasks. arXiv preprint arXiv:2103.05823.
- Kirchheim, K., Gonschorek, T., & Ortmeier, F. (2024). Out-of-distribution detection with logical reasoning. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision* (pp. 2122–2131).
- Körner, P., Leuschel, M., Barbosa, J., Costa, V. S., Dahl, V., Hermenegildo, M. V., Morales, J. F., Wielemaker, J., Diaz, D., Abreu, S., et al. (2022). Fifty years of prolog and beyond. *Theory and Practice of Logic Programming*, 22(6), 776–858.
- Lake, B. M., Ullman, T. D., Tenenbaum, J. B., & Gershman, S. J. (2017). Building machines that learn and think like people. *Behavioral and Brain Sciences*, 40, e253.
- LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436–444.
- Lee, S., Sim, W., Shin, D., Hwang, S., Seo, W., Park, J., Lee, S., Kim, S., & Kim, S. (2024). Reasoning abilities of large language models: In-depth analysis on the abstraction and reasoning corpus. arXiv preprint arXiv:2403.11793.
- Lei, C., Lipovetzky, N., & Ehinger, K. A. (2024). Generalized planning for the abstraction and reasoning corpus. In: *Proceedings of the Thirty-Eighth AAAI Conference on Artificial Intelligence and Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence and Fourteenth Symposium on Educational Advances in Artificial Intelligence*. AAAI'24/IAAI'24/EAAI'24, AAAI Press. <https://doi.org/10.1609/aaai.v38i18.29996>
- Li, W. D., Hu, K., Larsen, C., Wu, Y., Alford, S., Woo, C., Dunn, S. M., Tang, H., Naim, M., Nguyen, D., Zheng, W. L., Tavares, Z., Pu, Y., & Ellis, K. (2024). Combining induction and transduction for abstract reasoning. <https://arxiv.org/abs/2411.02272>
- Lin, D., Dechter, E., Ellis, K., Tenenbaum, J., & Muggleton, S. (2014). Bias reformulation for one-shot function induction. *Frontiers in Artificial Intelligence and Applications*, 263, 525–530. 10.3233/978-1-61499-419-0-525
- Marcus, G. (2018). Deep learning: A critical appraisal. arXiv preprint arXiv:1801.00631.
- McCoy, R. T., Yao, S., Friedman, D., Hardy, M., & Griffiths, T. L. (2023). Embers of autoregression: Understanding large language models through the problem they are trained to solve. arXiv preprint arXiv:2309.13638.
- Mitchell, M., Palmarini, A. B., & Moskvichev, A. (2023). Comparing humans, gpt-4, and gpt-4v on abstraction and reasoning tasks. arXiv preprint arXiv:2311.09247.
- Muggleton, S., Dai, W. Z., Sammut, C., Tamaddoni-Nezhad, A., Wen, J., & Zhou, Z. H. (2018). Meta-interpretive learning from noisy images. *Machine Learning*, 107, 1097–1118.
- Muggleton, S., & De Raedt, L. (1994). Inductive logic programming: Theory and methods. *The Journal of Logic Programming*, 19, 629–679.
- Naveed, H., Khan, A. U., Qiu, S., Saqib, M., Anwar, S., Usman, M., Barnes, N., & Mian, A. (2023). A comprehensive overview of large language models. arXiv preprint arXiv:2307.06435.
- Nezhurina, M., Cipolina-Kun, L., Cherti, M., & Jitsev, J. (2024). Alice in wonderland: Simple tasks showing complete reasoning breakdown in state-of-the-art large language models. <https://arxiv.org/abs/2406.02061>
- Quinlan, J. R. (1990). Learning logical definitions from relations. *Machine Learning*, 5, 239–266.
- Raedt, L. D. (2008). *Representations for Mining and Learning* (pp. 71–114). Springer Berlin Heidelberg, Berlin, Heidelberg.
- Reiter, R. (1978). *On Closed World Data Bases*. Springer. (pp. 55–76). 10.1007/978-1-4684-3384-5_3
- Shrestha, A., & Mahmood, A. (2019). Review of deep learning algorithms and architectures. *IEEE Access*, 7, 53040–53065.
- Singh, M., Cambronero, J., Gulwani, S., Le, V., & Verbruggen, G. (2023). Assessing gpt4-v on structured reasoning tasks. arXiv preprint arXiv:2312.11524.
- Spelke, E. S., & Kinzler, K. D. (2007). Core knowledge. *Developmental Science*, 10(1), 89–96.
- Witt, J., Rasing, S., Dumančić, S., Guns, T., & Carbon, C. C. (2023). A divide-align-conquer strategy for program synthesis. arXiv preprint arXiv:2301.03094.
- Wu, Z., Qiu, L., Ross, A., Akyürek, E., Chen, B., Wang, B., Kim, N., Andreas, J., & Kim, Y. (2023). Reasoning or reciting? exploring the capabilities and limitations of language models through counterfactual tasks. arXiv preprint arXiv:2307.02477.
- Xu, Y., Khalil, E. B., & Sanner, S. (2022). Graphs, constraints, and search for the abstraction and reasoning corpus. In *NeurIPS 2022 Workshop on Neuro Causal and Symbolic AI (nCSI)*. <https://openreview.net/forum?id=dqsRoGivU4R>
- Xu, Y., Li, W., Vaezipoor, P., Sanner, S., & Khalil, E. B. (2023). LLMs and the abstraction and reasoning corpus: Successes, failures, and the importance of object-based representations. arXiv preprint arXiv:2305.18354.
- Ye, N., Li, K., Bai, H., Yu, R., Hong, L., Zhou, F., Li, Z., & Zhu, J. (2022). Ood-bench: Quantifying and understanding two dimensions of out-of-distribution generalization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (pp. 7947–7958).